Medicine Stock Prediction Using Linear Regression and Random Forest Regression

(Hospital Pharmacy Medicine Inventory and Stock Requirement Prediction using Machine Learning)

Problem Statement:

Hospital pharmacy inventory management represents one of the most vital operational pillars in healthcare administration. Maintaining an optimal stock of life-saving medicines requires an intricate balance between preventing critical inventory stockouts and avoiding excessive overstocking. A shortage of essential pharmaceuticals—such as antibiotics, analgesics, insulin, respiratory treatments, and cardiovascular drugs—can directly compromise patient care, leading to clinical treatment interruptions, delayed therapeutic interventions, and costly emergency procurements. Conversely, excessive medicine inventory leads to substantial capital lockup, elevated storage and refrigeration expenses, and a high risk of drug expiration prior to dispensing.

In many healthcare institutions, medicine replenishment decisions still depend on manual heuristic estimations, periodic visual stock inspections, or static minimum-threshold rules that fail to capture historical consumption trends, patient burn rates, and supplier delivery delays. There is, therefore, an urgent need for an automated, data-driven, and accessible machine learning system capable of predicting future continuous medicine demand and recommending transparent inventory reorder quantities with clear safety stock buffers. This project designs and implements an end-to-end web-based Medicine Stock Prediction system that performs data preprocessing, feature encoding, model benchmarking using Linear Regression and Random Forest Regressor, 5-fold cross-validation on training data, and held-out test evaluation using regression metrics (MAE, MSE, RMSE, R^2).

Objectives:

1. To collect, validate, and inspect a structured pharmaceutical inventory dataset containing medicine usage, current stock, historical consumption, and distributor delivery lead times.
2. To perform thorough Exploratory Data Analysis (EDA) with statistical visualizations (histograms, scatter plots, category demand distributions, lead-time box plots, and correlation heatmaps).
3. To construct a robust Scikit-learn preprocessing pipeline using `StandardScaler` for numerical attributes and `OneHotEncoder(handle_unknown='ignore')` for categorical features, fitted strictly on training data.
4. To design, train, and compare baseline Linear Regression and ensemble Random Forest Regressor models using 5-fold cross-validation solely on the training partition.
5. To evaluate the champion model on a held-out test set and report Mean Absolute Error (MAE), Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and R-squared (R^2), paired with diagnostic residual plots.
6. To build a Flask-based backend and responsive web front-end that accepts medicine inventory parameters, generates real-time ML demand forecasts, and calculates recommended reorder quantities and stock status (Low Stock, Sufficient Stock, Overstock).

Software / Hardware Required:

- **Operating System:** Windows / Linux / macOS
- **IDE:** Visual Studio Code
- **Language:** Python 3.10+
- **Backend Framework:** Flask (REST API)

• **Machine Learning:** scikit-learn (LinearRegression, RandomForestRegressor, ColumnTransformer, Pipeline)
• **Model Persistence:** Joblib
• **Data Processing:** Pandas, NumPy
• **Data Visualization:** Matplotlib, Seaborn
• **Frontend:** HTML5, CSS3, JavaScript
• **Dataset:** 700-record Pharmaceutical Inventory Dataset (Simulated Educational Demonstration Dataset)
• **Version Control:** Git

Project Directory Structure:

The project ("MedicineStockPrediction") is organized as a Flask backend with a separate dataset, trained-model, static plots, and frontend template folder, as shown below.

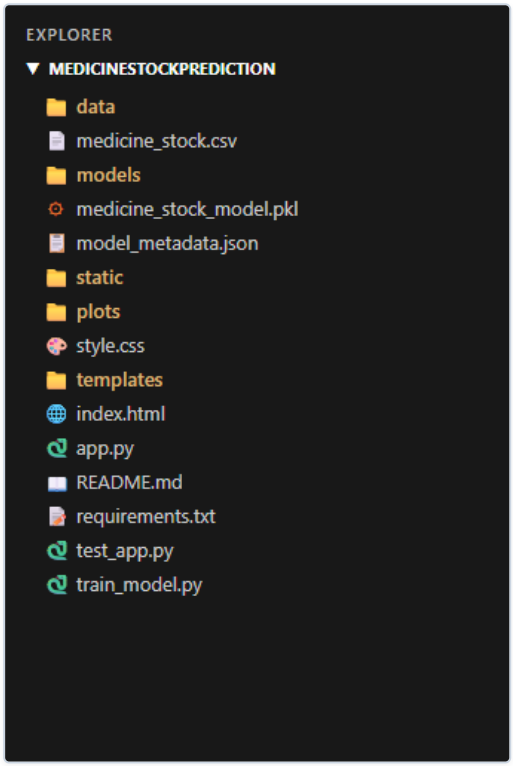


Fig 1: Project directory structure of MedicineStockPrediction in Visual Studio Code

Procedure:

1. Prepare and inspect the pharmaceutical inventory dataset (`data/medicine_stock.csv`) containing 700 records with 6 predictive features and continuous medicine demand.
2. Perform data validation to verify zero missing values, zero duplicate entries, valid data types, and non-negativity across all numerical variables.
3. Conduct Exploratory Data Analysis (EDA) and generate 6 visual plots in `static/plots/` to examine demand distributions, category-level usage, delivery lead times, and feature correlations.
4. Partition the dataset into 80% training data (560 samples) and 20% held-out test data (140 samples) using `train_test_split(test_size=0.2, random_state=42)`.
5. Build a Scikit-learn `ColumnTransformer` pipeline incorporating `StandardScaler` for numeric features and `OneHotEncoder(handle_unknown='ignore')` for categorical features, fitted exclusively on training data.
6. Train both Linear Regression and Random Forest Regressor models using 5-fold cross-validation on the training set to evaluate generalization error.
7. Select the champion algorithm based on the lowest cross-validation RMSE (Linear Regression achieved 35.15 units).
8. Evaluate the selected model pipeline on the held-out test set, reporting MAE, MSE, RMSE, and R^2, and generate Actual vs. Predicted and Residual diagnostic plots.
9. Serialize the complete fitted pipeline to `models/medicine_stock_model.pkl` and export metadata to `models/model_metadata.json`.
10. Launch the Flask web server (`app.py`), process user input through the prediction form, execute pipeline inference, and compute inventory replenishment metrics (safety stock, target stock, reorder qty, stock status).

Implementation Code:

Representative implementation of the key modules used in the project, based on the pipeline shown in the application.

1. Data Preprocessing & ColumnTransformer Pipeline (train_model.py)

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer

df = pd.read_csv("data/medicine_stock.csv")
feature_cols = ["Medicine Name", "Medicine Category", "Current Stock",
                "Previous Consumption", "Average Daily Usage", "Supplier Delivery Time"]
X, y = df[feature_cols], df["Predicted Demand"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, random_state=42)

numeric_features = ["Current Stock", "Previous Consumption", "Average Daily Usage", "Supplier
Delivery Time"]
categorical_features = ["Medicine Name", "Medicine Category"]

preprocessor = ColumnTransformer(transformers=[
    ("num", StandardScaler(), numeric_features),
    ("cat", OneHotEncoder(handle_unknown="ignore", sparse_output=False), categorical_features)
])
```

2. Model Training & 5-Fold Cross-Validation (train_model.py)

```
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import KFold, cross_validate
from sklearn.pipeline import Pipeline
import joblib, numpy as np

models = {
    "Linear Regression": LinearRegression(),
    "Random Forest Regressor": RandomForestRegressor(n_estimators=100, random_state=42)
}

cv = KFold(n_splits=5, shuffle=True, random_state=42)
cv_results = {}

for name, model in models.items():
    pipeline = Pipeline(steps=[("preprocessor", preprocessor), ("regressor", model)])
    scores = cross_validate(pipeline, X_train, y_train, cv=cv,
                            scoring=["neg_root_mean_squared_error", "r2"])
    cv_results[name] = -np.mean(scores["test_neg_root_mean_squared_error"])

# Select champion model based on lowest cross-validation RMSE
best_model_name = min(cv_results, key=cv_results.get)
final_pipeline = Pipeline(steps=[("preprocessor", preprocessor), ("regressor",
models[best_model_name])])
final_pipeline.fit(X_train, y_train)
joblib.dump(final_pipeline, "models/medicine_stock_model.pkl")
```

3. Test Set Evaluation & Residual Calculations (train_model.py)

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

y_pred = final_pipeline.predict(X_test)
y_pred = np.clip(y_pred, 0, None)  # Enforce non-negativity

mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print(f"Test MAE: {mae:.2f} | RMSE: {rmse:.2f} | R2: {r2:.4f}")
```

4. Inventory Replenishment Policy Calculation (app.py)

```
import math

def calculate_inventory_policy(predicted_demand, current_stock, daily_usage, delivery_time):
    # Safety stock buffer protecting against lead time demand shocks
    safety_stock = int(math.ceil(daily_usage * math.ceil(delivery_time * 0.5)))
    recommended_stock = int(math.ceil(predicted_demand + safety_stock))
    reorder_quantity = max(0, recommended_stock - int(current_stock))

    if current_stock < (predicted_demand * 0.5):
        stock_status = "Low Stock"
    elif current_stock > (recommended_stock * 1.25):
        stock_status = "Overstock"
    else:
        stock_status = "Sufficient Stock"

    return {
        "safety_stock": safety_stock, "recommended_stock": recommended_stock,
        "reorder_quantity": reorder_quantity, "stock_status": stock_status
    }
```

5. Flask Prediction Endpoint (app.py)

```
from flask import Flask, render_template, request
import joblib, pandas as pd

app = Flask(__name__)
model_pipeline = joblib.load("models/medicine_stock_model.pkl")

@app.route("/", methods=["GET", "POST"])
def index():
    prediction_result = None
    if request.method == "POST":
        input_df = pd.DataFrame([{
            "Medicine Name": request.form.get("medicine_name"),
            "Medicine Category": request.form.get("medicine_category"),
            "Current Stock": int(round(float(request.form.get("current_stock")))),
            "Previous Consumption": int(round(float(request.form.get("previous_consumption")))),
            "Average Daily Usage": float(request.form.get("average_daily_usage")),
            "Supplier Delivery Time": int(round(float(request.form.get("supplier_delivery_time"))))
        }])

        pred_raw = model_pipeline.predict(input_df)[0]
        predicted_demand = max(0, int(round(pred_raw)))
        policy = calculate_inventory_policy(predicted_demand, input_df["Current Stock"][0],
                                            input_df["Average Daily Usage"][0],
                                            input_df["Supplier Delivery Time"][0])
        prediction_result = {"predicted_demand": predicted_demand, "policy": policy}

    return render_template("index.html", prediction_result=prediction_result)
```

Output (Step-by-Step Screenshots):

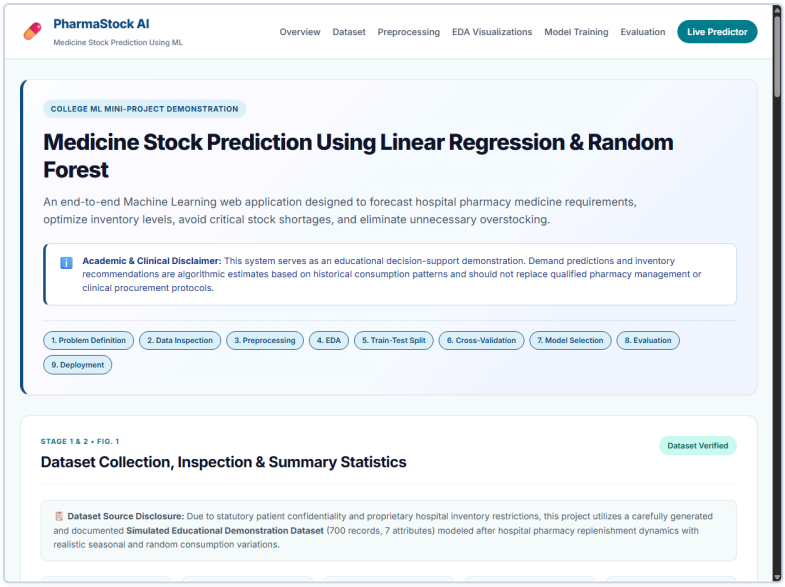


Fig 2: Step 1 — Load Interface: Web application dashboard with project overview and dataset summary

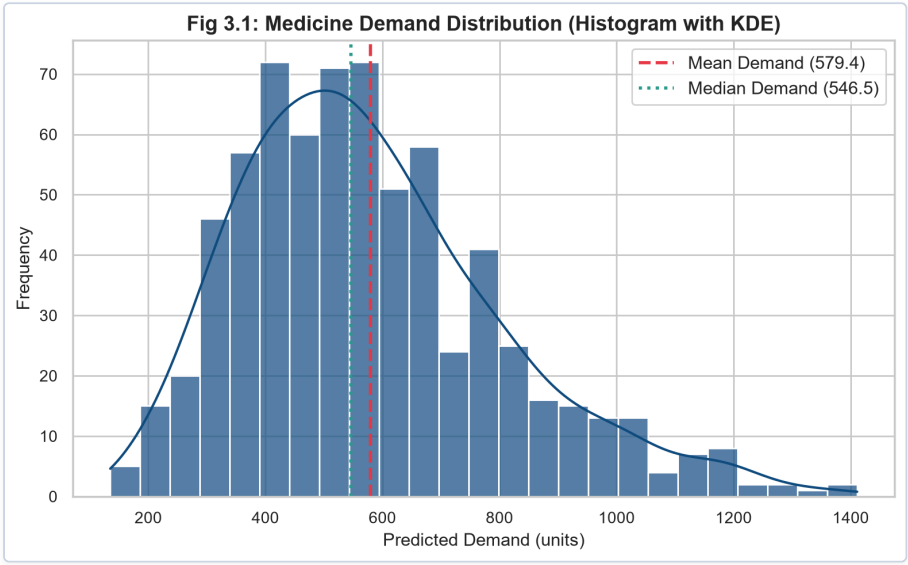


Fig 3: Step 2 — Exploratory Data Analysis: Medicine demand distribution histogram with Kernel Density Estimation

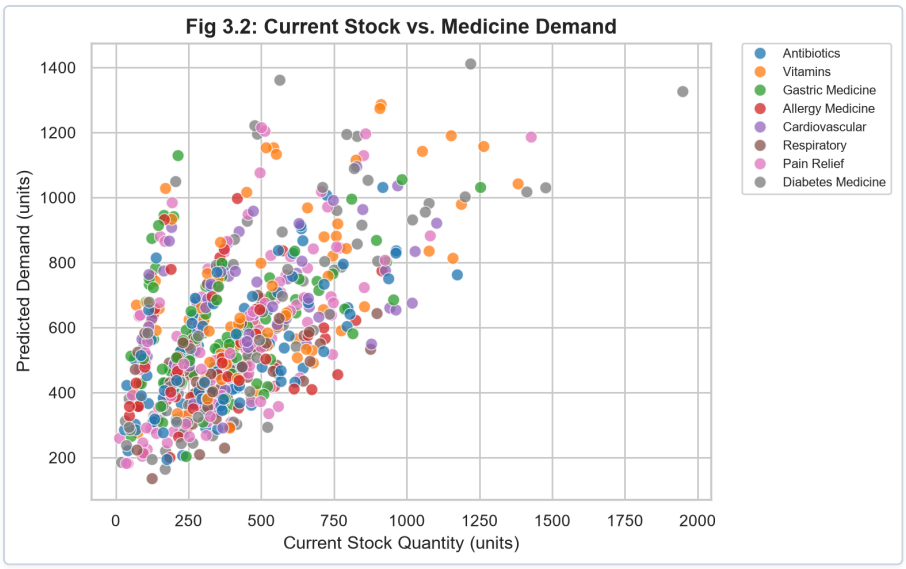


Fig 4: Step 2 — Exploratory Data Analysis: Current stock quantity vs. medicine demand categorized by therapeutic class

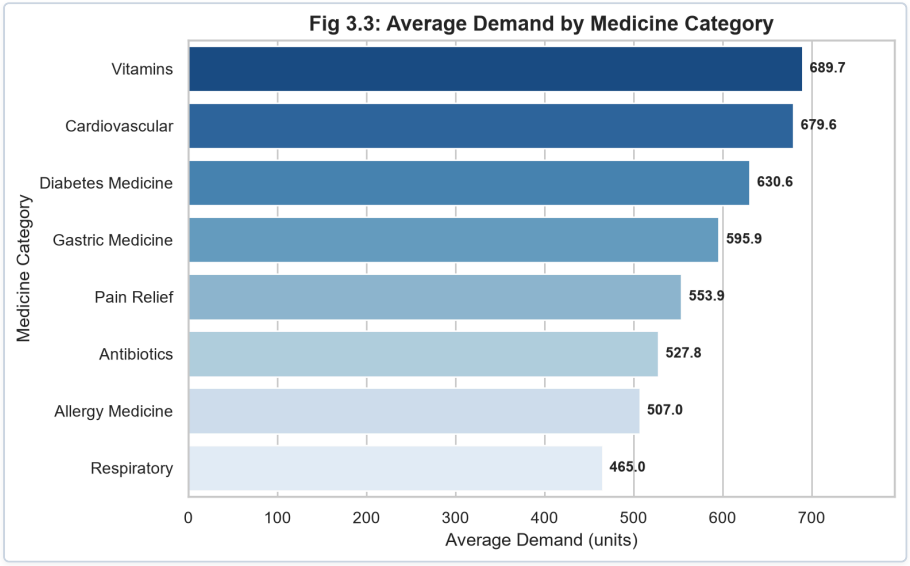


Fig 5: Step 2 — Exploratory Data Analysis: Average medicine demand comparison across therapeutic categories

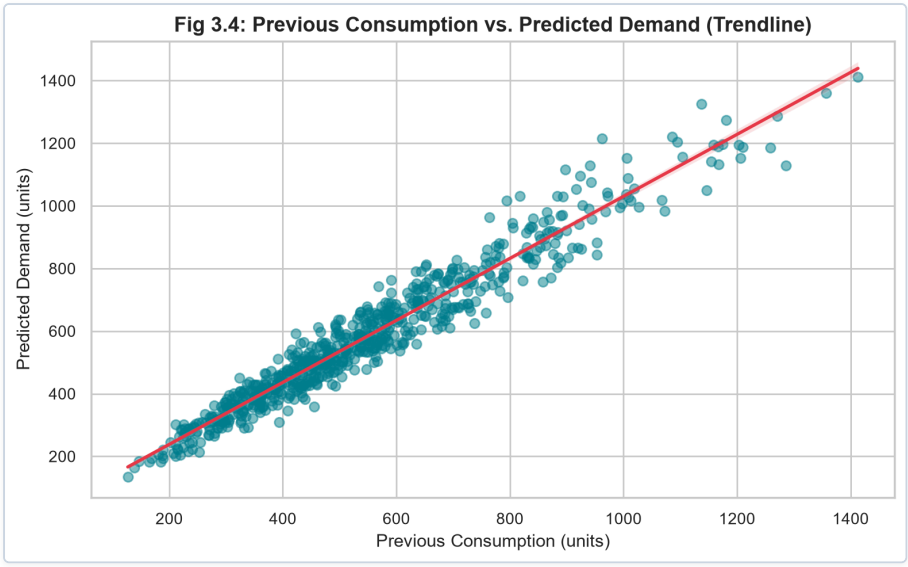


Fig 6: Step 2 — Exploratory Data Analysis: Previous consumption vs. predicted demand with linear regression trendline

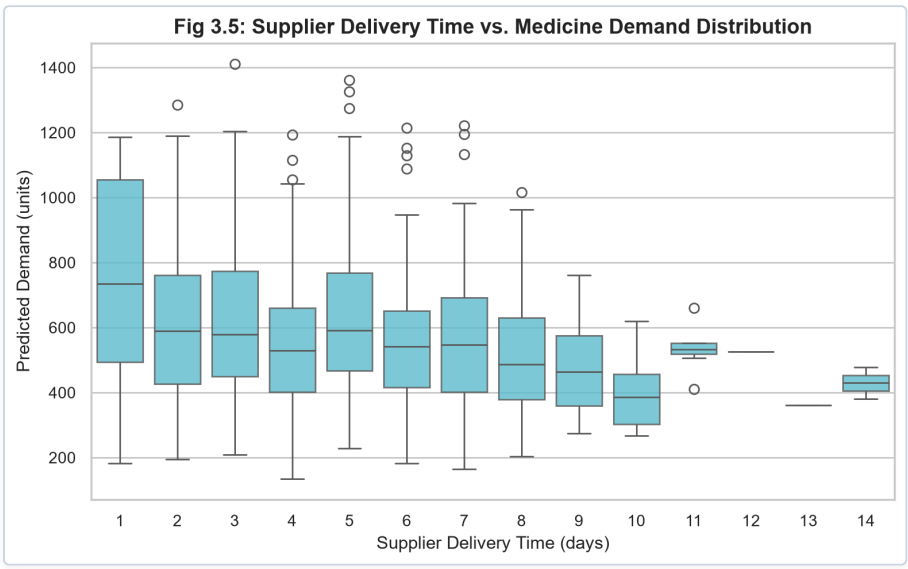


Fig 7: Step 2 — Exploratory Data Analysis: Supplier delivery time (days) vs. medicine demand distribution box plot

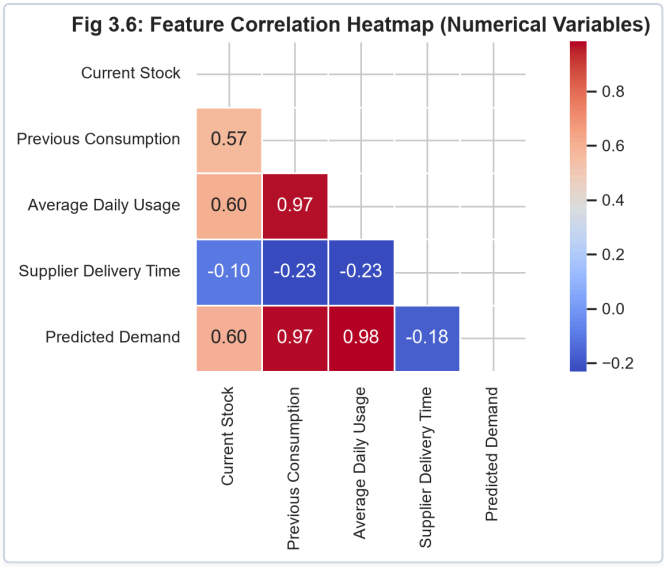


Fig 8: Step 2 — Exploratory Data Analysis: Pearson correlation matrix across numerical inventory features

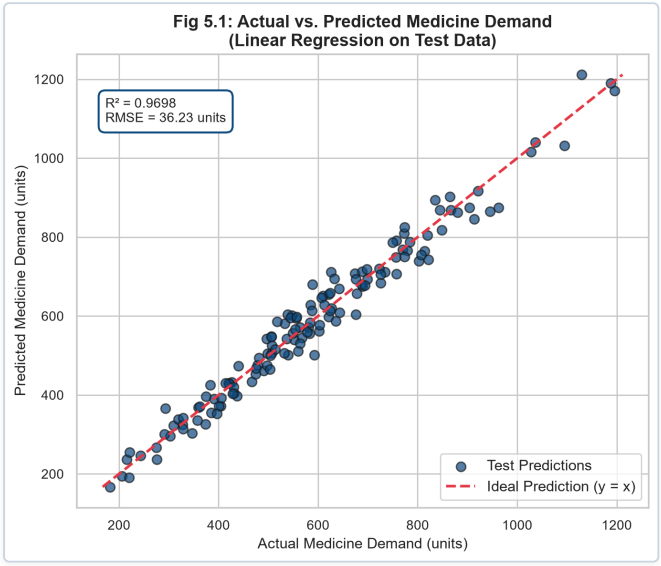


Fig 9: Step 3 — Model Evaluation: Actual vs. predicted medicine demand on held-out test data with ideal y = x line

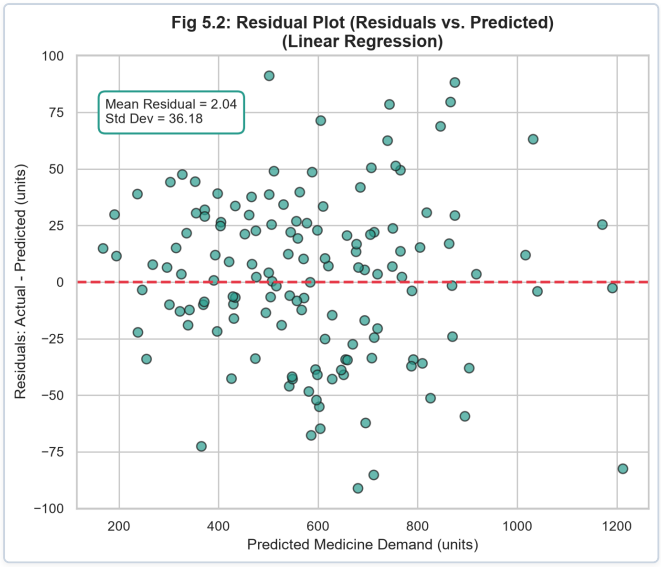


Fig 10: Step 4 — Diagnostic Residual Analysis: Residual plot (Residuals vs. Predicted Demand) verifying zero-centered error

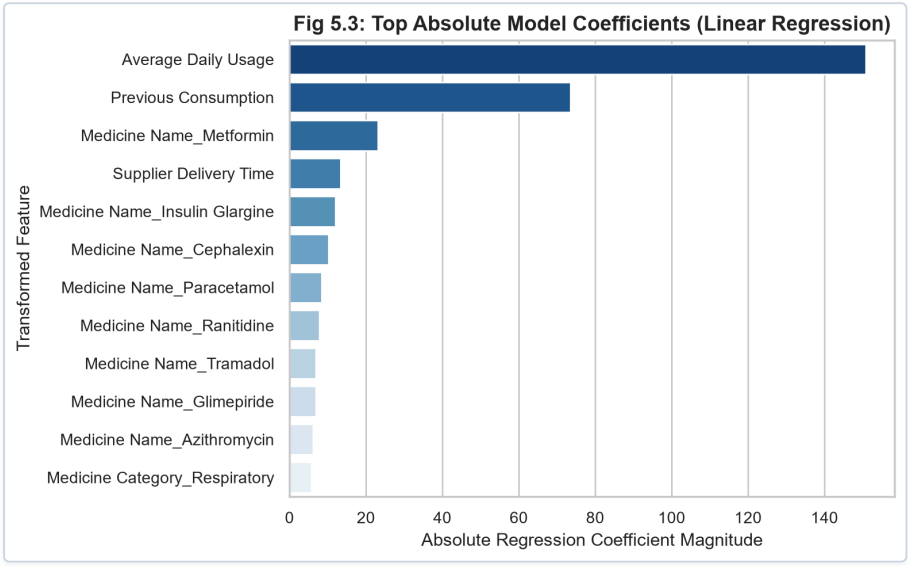


Fig 11: Step 4 — Model Feature Impact: Top absolute regression coefficients for standardized predictive features

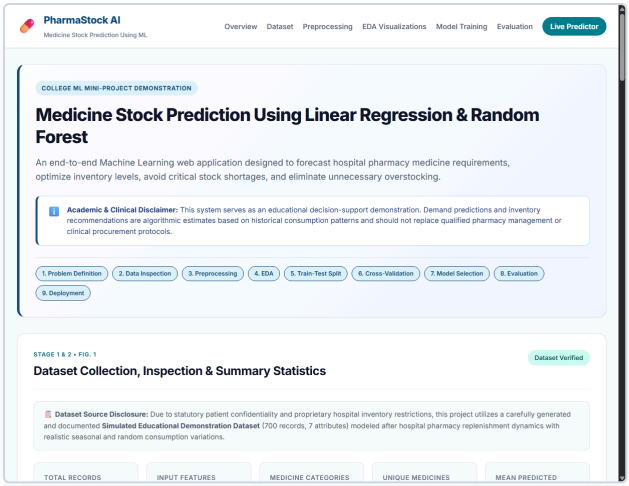


Fig 12: Step 5 — Medicine Stock Prediction: Low Stock status with recommended reorder quantity calculation (Paracetamol)

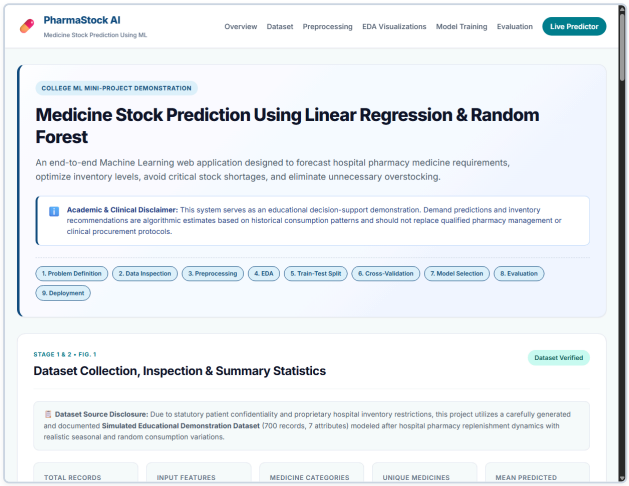


Fig 13: Step 5 — Medicine Stock Prediction: Sufficient Stock status indicating balanced inventory (Amoxicillin)

Result:

A web-based Medicine Stock Prediction system was successfully designed and implemented using a Flask backend, Scikit-learn regression pipelines (Linear Regression and Random Forest Regressor), and an automated inventory replenishment policy. The pharmaceutical inventory dataset (700 records, 6 features) was preprocessed with zero data leakage using a `ColumnTransformer` pipeline applying `StandardScaler` to numeric attributes and `OneHotEncoder` to categorical fields. In 5-fold cross-validation on the training set, Linear Regression achieved superior performance (CV RMSE of 35.15 ± 0.82 units, CV R^2 of 97.62%) and was selected over Random Forest Regressor (CV RMSE of 40.94 ± 2.24 units). On the 20% held-out test set, the champion model achieved a Mean Absolute Error (MAE) of 28.89 units, a Root Mean Squared Error (RMSE) of 36.24 units, and an R-squared (R^2) score of 96.98%. The model is deployed via an interactive Flask web application that generates real-time demand forecasts, calculates safety stock buffers and reorder quantities, and categorizes inventory status into Low Stock, Sufficient Stock, or Overstock for clinical pharmacy decision support.